

Jobsheet Praktikum: Pertemuan 6

Inheritance (Pewarisan)

Mata Kuliah	Praktikum Pemrograman Berbasis Objek (RTI253008)
Pertemuan	6 (Minggu 6)
Durasi	1 × 4 × 50' praktikum; 1 × 1 × 50' tugas/laporan mandiri
Sub-CPMK	Mengimplementasikan pewarisan (is-a): single dan multilevel inheritance, hak akses member, kata kunci this dan super, konstruktor, dan overriding sederhana dalam kode Java
Kode Awal	tidak ada prasyarat kode wajib; tiap percobaan dimulai dari package baru yang berdiri sendiri
Kode Akhir	proyek berisi 5 package <code>id.ac.polinema.inheritance.percobaan1</code> s.d. <code>percobaan5</code> , ditambah package tugas mandiri

A. Capaian Praktikum

Setelah menyelesaikan jobsheet ini, mahasiswa mampu:

1. Menjelaskan konsep **inheritance**, istilah **superclass** dan **subclass**, serta kapan pewarisan tepat dipakai (hubungan **is-a**).
2. Membuat subclass dengan kata kunci `extends` untuk **single** dan **multilevel inheritance**.
3. Memilih access modifier (`private`, `protected`, `public`) yang tepat dan menjelaskan dampaknya terhadap member yang dapat diakses subclass.
4. Menggunakan kata kunci `this` dan `super` untuk merujuk member class sendiri dan member superclass.
5. Memanggil konstruktor superclass dengan `super(...)`, serta menjelaskan **urutan eksekusi konstruktor** pada hierarki pewarisan.
6. Menerapkan **overriding** sederhana dengan `@Override` dan `super.namaMethod()`.

B. Persiapan dan Prasyarat

- **Alat:** JDK 17 atau lebih baru, VSCode (editor yang kita pakai sepanjang praktikum ini). Kode pada jobsheet ini sudah diuji dengan JDK 21.
- **Prasyarat:** sudah menyelesaikan jobsheet kelas dan objek serta relasi kelas (atribut, konstruktor, method, access modifier, getter/setter, dan relasi has-a pada Jobsheet 04).
- **Proyek:** setiap percobaan memakai package baru bernama `id.ac.polinema.inheritance.percobaanN`. Package terpisah untuk tiap percobaan supaya nama kelas yang sama (mis. `ClassA`, `ClassB`, dan `ClassC` pada Percobaan 1, 2, dan 4) tidak saling bentrok.



Info: Konvensi package sama seperti jobsheet sebelumnya: `id.ac.polinema.inheritance.percobaanN`. Satu file hanya berisi satu class `public`, dan nama file harus sama dengan nama class.

Peta percobaan yang akan dipraktikkan, supaya mahasiswa tahu ke mana arah jobsheet ini, berikut ringkasannya sebelum mulai:

Percobaan	Studi Kasus	Topik	Konsep Baru
1	ClassA – ClassB	Single inheritance	<code>extends</code> , member yang diwarisi
2	ClassA – ClassB	Hak akses	<code>private</code> vs <code>protected</code> , getter sebagai alternatif
3	Bangun – Tabung	<code>this</code> dan <code>super</code>	akses atribut superclass, bayangan atribut (shadowing)
4	ClassA – ClassB – ClassC	Konstruktor dan multilevel inheritance	urutan konstruktor, <code>super()</code> harus baris pertama
5	Komputer – Desktop – Laptop	Konstruktor berparameter dan overriding	<code>super(...)</code> , <code>@Override</code> , <code>super.showInfo()</code>

C. Langkah Kerja

Setiap Percobaan di bawah ini berdiri sendiri (package terpisah) dan dipecah menjadi langkah-langkah kecil. Kerjakan berurutan, tiap langkah dibangun di atas langkah sebelumnya di Percobaan yang sama.

Percobaan 1: Single Inheritance dengan `extends` (ClassA dan ClassB)

Inheritance (pewarisan) adalah cara menurunkan sebuah class yang lebih umum (**superclass**) menjadi class yang lebih spesifik (**subclass**). Subclass otomatis memiliki atribut dan method superclass, lalu boleh menambahkan miliknya sendiri. Inilah inti sifat *reusable* pada pemrograman berorientasi objek. Pada percobaan ini **ClassB** akan diturunkan dari **ClassA**, sehingga objek ClassB dapat memakai `x`, `y`, dan `getNilai()` tanpa menulis ulang.

Langkah 1: Diagram kelas

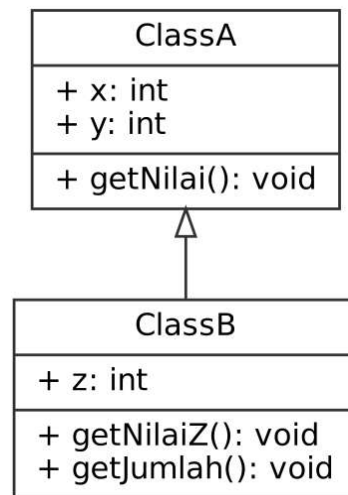


Figure 1: Diagram kelas ClassA dan ClassB, segitiga kosong di sisi ClassA menandakan ClassA adalah superclass dan ClassB adalah subclass.

Catatan Konsep: Segitiga kosong Δ = **Inheritance** (generalization). Sisi yang dekat segitiga adalah **superclass**, ujung lainnya **subclass**. Di Java ditulis dengan **extends**. Subclass mewarisi seluruh member superclass **kecuali** member **private** dan **konstruktor**.

Info: Istilah lain: superclass = *parent class* / *base class*; subclass = *child class* / *derived class*. Java hanya mengizinkan **satu** superclass langsung (*single inheritance*), tetapi satu superclass boleh memiliki banyak subclass.

Langkah 2: Buat class ClassA

Buat package `id.ac.polinema.inheritance.percobaan1`, lalu buat class `ClassA` dengan dua atribut `public` dan method `getNilai()`:

```
1 package id.ac.polinema.inheritance.percobaan1;
2
3 public class ClassA {
4     public int x;
5     public int y;
6
7     public void getNilai() {
8         System.out.println("nilai x: " + x);
9         System.out.println("nilai y: " + y);
10    }
11 }
```

Kode 1: ClassA.java: atribut x dan y, serta getNilai()

Langkah 3: Buat class ClassB (belum memakai extends)

Pada package yang sama, buat class `ClassB`. **Perhatikan:** untuk sementara ClassB sengaja belum diturunkan dari ClassA.

```
1 package id.ac.polinema.inheritance.percobaan1;
2
3 public class ClassB {
4     public int z;
```

```

5
6     public void getNilaiZ() {
7         System.out.println("nilai z: " + z);
8     }
9
10    public void getJumlah() {
11        System.out.println("jumlah: " + (x + y + z));
12    }
13 }

```

Kode 2: ClassB.java: belum ada `extends`, tetapi `getJumlah()` memakai x dan y

Langkah 4: Buat MainPercobaan1

Buat class **MainPercobaan1** yang membuat objek **ClassB** lalu mengisi dan menampilkan nilainya:

```

1 package id.ac.polinema.inheritance.percobaan1;
2
3 public class MainPercobaan1 {
4     public static void main(String[] args) {
5         ClassB hitung = new ClassB();
6         hitung.x = 20;
7         hitung.y = 30;
8         hitung.z = 5;
9         hitung.getNilai();
10        hitung.getNilaiZ();
11        hitung.getJumlah();
12    }
13 }

```

Kode 3: MainPercobaan1.java

Langkah 5: Kompilasi dan amati error

Kompilasi program (atau lihat garis merah di VSCode). Kompilasi **gagal**, dan itu memang diharapkan pada langkah ini.

Catatan Konsep: Pesan error yang muncul kurang lebih berbunyi **cannot find symbol ... symbol: variable x ... location: class ClassB**. Salin pesan error pertama beserta nama file dan nomor barisnya, karena akan ditanyakan pada pertanyaan nomor 1.

Langkah 6: Perbaiki dengan extends

Ubah deklarasi **ClassB** agar menjadi subclass dari **ClassA**:

```

1 package id.ac.polinema.inheritance.percobaan1;
2
3 public class ClassB extends ClassA {
4     public int z;
5
6     public void getNilaiZ() {
7         System.out.println("nilai z: " + z);
8     }
9
10    public void getJumlah() {
11        System.out.println("jumlah: " + (x + y + z));
12    }
13 }

```

Kode 4: ClassB.java: ditambah `extends ClassA` pada baris deklarasi class

Langkah 7: Jalankan ulang

✓ **Checkpoint:** output menampilkan
nilai x: 20, nilai y: 30, nilai z: 5
dan jumlah: 55.

Pertanyaan Percobaan 1

1. Mengapa kompilasi pada Langkah 5 gagal? Tuliskan pesan error pertama beserta file dan baris tempat error muncul.
2. Baris kode mana yang diubah pada Langkah 6, dan apa artinya? Sebutkan class yang berperan sebagai **superclass** dan **subclass**.
3. Setelah diperbaiki, sebutkan atribut dan method yang dapat dipakai oleh objek **hitung**. Kelompokkan mana yang dideklarasikan di **ClassA** dan mana yang dideklarasikan di **ClassB**.
4. Pada **MainPercobaan1**, **hitung.x = 20** ditulis pada objek **ClassB**, padahal atribut **x** tidak dideklarasikan di **ClassB**. Mengapa hal ini diperbolehkan?
5. Atribut **x** dan **y** pada **ClassA** dibuat **public**, sehingga dapat diubah langsung dari **MainPercobaan1**. Apa risiko dari desain seperti ini? (jawaban ini akan kita telusuri pada Percobaan 2)
6. Coba tambahkan class **ClassD** lalu ubah deklarasi menjadi **public class ClassB extends ClassA, ClassD**. Apa yang terjadi? Apa yang dapat Anda simpulkan tentang jumlah superclass langsung pada Java?

Percobaan 2: Hak Akses pada Pewarisan (private dan protected)

Tidak semua member superclass dapat dipakai oleh subclass. Member yang dideklarasikan **private** **tidak diwariskan**, sehingga tidak dapat diakses langsung dari subclass. Percobaan ini membuktikannya lewat error kompilasi, lalu memperbaikinya dengan dua cara.

Langkah 1: Tabel kontrol pengaksesan

Modifier	Class yang sama	Package yang sama	Subclass (beda package)	Class mana pun
private	✓			
<i>default</i> (tanpa modifier)	✓	✓		
protected	✓	✓	✓	
public	✓	✓	✓	✓

■ **Catatan Konsep:** Gunakan **protected** bila sebuah member perlu diakses oleh subclass tetapi tetap disembunyikan dari class lain di luar package. Gunakan **private** bila member hanya boleh dipakai oleh class itu sendiri, dan sediakan getter/setter bila class lain perlu mengaksesnya secara terkontrol.

Langkah 2: Buat class ClassA dengan atribut private

Buat package `id.ac.polinema.inheritance.percobaan2`, lalu buat class `ClassA`. Kali ini atribut `x` dan `y` dibuat `private`, dan nilainya diisi lewat setter:

```
1 package id.ac.polinema.inheritance.percobaan2;
2
3 public class ClassA {
4     private int x;
5     private int y;
6
7     public void setX(int x) {
8         this.x = x;
9     }
10
11     public void setY(int y) {
12         this.y = y;
13     }
14
15     public void getNilai() {
16         System.out.println("nilai x: " + x);
17         System.out.println("nilai y: " + y);
18     }
19 }
```

Kode 5: ClassA.java: atribut private dengan setter

Langkah 3: Buat class ClassB

Buat `ClassB` sebagai subclass dari `ClassA`. Method `getJumlah()` mengakses `x` dan `y` secara langsung:

```
1 package id.ac.polinema.inheritance.percobaan2;
2
3 public class ClassB extends ClassA {
4     private int z;
5
6     public void setZ(int z) {
7         this.z = z;
8     }
9
10    public void getNilaiZ() {
11        System.out.println("nilai z: " + z);
12    }
13
14    public void getJumlah() {
15        System.out.println("jumlah: " + (x + y + z));
16    }
17 }
```

Kode 6: ClassB.java: getJumlah() memakai x dan y milik ClassA

Langkah 4: Buat MainPercobaan2

```
1 package id.ac.polinema.inheritance.percobaan2;
2
3 public class MainPercobaan2 {
4     public static void main(String[] args) {
5         ClassB hitung = new ClassB();
6         hitung.setX(20);
7         hitung.setY(30);
8         hitung.setZ(5);
9         hitung.getNilai();
10        hitung.getNilaiZ();
11        hitung.getJumlah();
12    }
13 }
```

```

12     }
13 }

```

Kode 7: MainPercobaan2.java: hanya memakai setter dan method public

Langkah 5: Kompilasi dan amati error

Kompilasi program. Kompilasi **gagal**, dan itu memang diharapkan pada langkah ini.

❏ **Catatan Konsep:** Error yang muncul berbunyi `x has private access in ClassA` dan `y has private access in ClassA`, dengan lokasi di **ClassB.java baris 15**, bukan di **MainPercobaan2**. Perhatikan bahwa **MainPercobaan2** sendiri hanya memanggil method **public**.

Langkah 6: Perbaiki A, ubah menjadi protected

Ubah modifier atribut `x` dan `y` pada **ClassA** menjadi **protected**, lalu jalankan **MainPercobaan2**:

```

1 package id.ac.polinema.inheritance.percobaan2;
2
3 public class ClassA {
4     protected int x;
5     protected int y;
6
7     public void setX(int x) {
8         this.x = x;
9     }
10
11     public void setY(int y) {
12         this.y = y;
13     }
14
15     public void getNilai() {
16         System.out.println("nilai x: " + x);
17         System.out.println("nilai y: " + y);
18     }
19 }

```

Kode 8: ClassA.java (Perbaikan A): atribut menjadi protected

✅ **Checkpoint:** kompilasi berhasil dan output menampilkan
nilai x: 20, nilai y: 30, nilai z: 5
dan jumlah: 55.

Langkah 7: Perbaiki B, kembali ke private dan gunakan getter

Kembalikan `x` dan `y` menjadi **private**. Tambahkan **getX()** dan **getY()** pada **ClassA**, lalu ubah **getJumlah()** pada **ClassB** agar memakai getter tersebut:

```

1 package id.ac.polinema.inheritance.percobaan2;
2
3 public class ClassA {
4     private int x;
5     private int y;
6
7     public void setX(int x) {
8         this.x = x;
9     }
10

```

```

11     public void setY(int y) {
12         this.y = y;
13     }
14
15     public void getNilai() {
16         System.out.println("nilai x: " + x);
17         System.out.println("nilai y: " + y);
18     }
19
20     public int getX() {
21         return x;
22     }
23
24     public int getY() {
25         return y;
26     }
27 }

```

Kode 9: ClassA.java (Perbaikan B): atribut private dengan getter

```

1 package id.ac.polinema.inheritance.percobaan2;
2
3 public class ClassB extends ClassA {
4     private int z;
5
6     public void setZ(int z) {
7         this.z = z;
8     }
9
10    public void getNilaiZ() {
11        System.out.println("nilai z: " + z);
12    }
13
14    public void getJumlah() {
15        System.out.println("jumlah: " + (getX() + getY() + z));
16    }
17 }

```

Kode 10: ClassB.java (Perbaikan B): getJumlah() memakai getX() dan getY()

☒ **Checkpoint:** output sama persis dengan Perbaikan A (**jumlah: 55**), tetapi atribut **x** dan **y** tetap tersembunyi dari class lain.

Pertanyaan Percobaan 2

1. Di file dan baris mana error pada Langkah 5 muncul, dan apa pesannya? Mengapa error tidak muncul di **MainPercobaan2**?
2. Jelaskan penyebab error tersebut dengan merujuk pada tabel kontrol pengaksesan (Langkah 1).
3. Pada kode awal, **MainPercobaan2** memanggil **hitung.setX(20)** dan tidak error, padahal **x** bersifat **private**. Mengapa pemanggilan ini diperbolehkan, dan di mana nilai **x** tersimpan?
4. Bandingkan Perbaikan A (**protected**) dan Perbaikan B (**private** + getter) dari sisi **encapsulation**. Mana yang Anda pilih untuk program sungguhan? Jelaskan alasannya.
5. Andaikan **ClassA** dan **ClassB** berada di **package yang berbeda**. Berdasarkan tabel, apakah **ClassB** tetap dapat mengakses atribut **protected** milik **ClassA**? Bagaimana jika atributnya **default** (tanpa modifier)?

Percobaan 3: Kata Kunci `this` dan `super` (Bangun dan Tabung)

Kata kunci `this` merujuk pada member **class itu sendiri**, sedangkan `super` merujuk pada member **superclass**. Pada percobaan ini class `Tabung` menurunkan `Bangun`, lalu memakai atribut `phi` dan `r` milik `Bangun` untuk menghitung volume.

Langkah 1: Diagram kelas

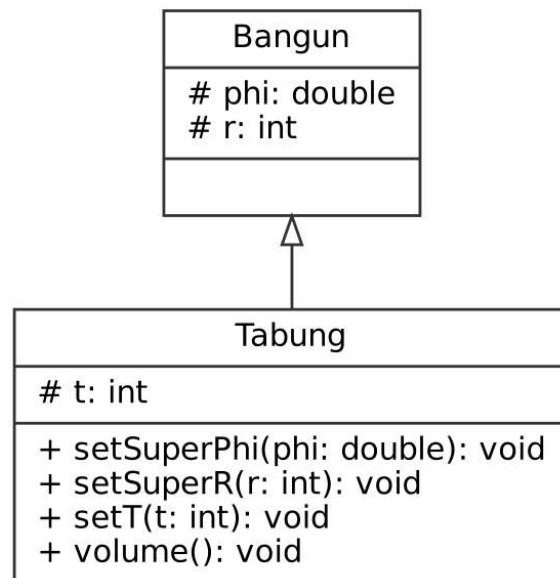


Figure 2: Diagram kelas `Bangun` dan `Tabung`. Atribut `phi` dan `r` bersifat `protected` agar dapat dipakai oleh `Tabung`.

Langkah 2: Buat class `Bangun`

Buat package `id.ac.polinema.inheritance.percobaan3`, lalu buat class `Bangun` dengan dua atribut `protected`:

```
1 package id.ac.polinema.inheritance.percobaan3;
2
3 public class Bangun {
4     protected double phi;
5     protected int r;
6 }
```

Kode 11: `Bangun.java`: atribut `phi` dan `r` bersifat `protected`

Langkah 3: Buat class `Tabung`

Buat `Tabung` sebagai subclass dari `Bangun`. Perhatikan pemakaian `super` dan `this` pada kode berikut:

```
1 package id.ac.polinema.inheritance.percobaan3;
2
3 public class Tabung extends Bangun {
4     protected int t;
5
6     public void setSuperPhi(double phi) {
7         super.phi = phi;
8     }
9
10    public void setSuperR(int r) {
11        super.r = r;
```

```

12     }
13
14     public void setT(int t) {
15         this.t = t;
16     }
17
18     public void volume() {
19         System.out.println("Volume Tabung adalah: "
20             + (super.phi * super.r * super.r * this.t));
21     }
22 }

```

Kode 12: Tabung.java: super merujuk atribut Bangun, this merujuk atribut Tabung

Langkah 4: Buat MainPercobaan3 lalu jalankan

```

1 package id.ac.polinema.inheritance.percobaan3;
2
3 public class MainPercobaan3 {
4     public static void main(String[] args) {
5         Tabung tabung = new Tabung();
6         tabung.setSuperPhi(3.14);
7         tabung.setSuperR(10);
8         tabung.setT(3);
9         tabung.volume();
10    }
11 }

```

Kode 13: MainPercobaan3.java

☒ **Checkpoint:** output menampilkan **Volume Tabung adalah: 942.0** ($= 3,14 \times 10 \times 10 \times 3$).

Langkah 5: Eksperimen 1, ganti super.phi dengan this.phi

Pada **Tabung**, ganti semua **super.phi** menjadi **this.phi**, lalu jalankan kembali. Setelah selesai, kembalikan ke **super.phi**.

```

1 package id.ac.polinema.inheritance.percobaan3;
2
3 public class Tabung extends Bangun {
4     protected int t;
5
6     public void setSuperPhi(double phi) {
7         this.phi = phi;
8     }
9
10    public void setSuperR(int r) {
11        super.r = r;
12    }
13
14    public void setT(int t) {
15        this.t = t;
16    }
17
18    public void volume() {
19        System.out.println("Volume Tabung adalah: "
20            + (this.phi * super.r * super.r * this.t));
21    }
22 }

```

Kode 14: Tabung.java (Eksperimen 1): this.phi menggantikan super.phi

✓ **Checkpoint:** output tetap **Volume Tabung** adalah: 942.0.

Langkah 6: Eksperimen 2, atribut dengan nama sama

Tambahkan atribut `protected int r = 5;` serta method `cekR()` pada `Tabung`, lalu panggil `tabung.cekR();` di akhir `main`:

```
1 package id.ac.polinema.inheritance.percobaan3;
2
3 public class Tabung extends Bangun {
4     protected int t;
5     protected int r = 5;
6
7     public void setSuperPhi(double phi) {
8         super.phi = phi;
9     }
10
11     public void setSuperR(int r) {
12         super.r = r;
13     }
14
15     public void setT(int t) {
16         this.t = t;
17     }
18
19     public void volume() {
20         System.out.println("Volume Tabung adalah: "
21             + (super.phi * super.r * super.r * this.t));
22     }
23
24     public void cekR() {
25         System.out.println("r          = " + r);
26         System.out.println("this.r   = " + this.r);
27         System.out.println("super.r  = " + super.r);
28     }
29 }
```

Kode 15: `Tabung.java` (Eksperimen 2): atribut `r` pada `Tabung` dan method `cekR()`

```
1 package id.ac.polinema.inheritance.percobaan3;
2
3 public class MainPercobaan3 {
4     public static void main(String[] args) {
5         Tabung tabung = new Tabung();
6         tabung.setSuperPhi(3.14);
7         tabung.setSuperR(10);
8         tabung.setT(3);
9         tabung.volume();
10        tabung.cekR();
11    }
12 }
```

Kode 16: `MainPercobaan3.java` (Eksperimen 2): memanggil `cekR()`

✓ **Checkpoint:** setelah baris `volume`, muncul tiga baris:

`r = 5, this.r = 5, dan super.r = 10.`

■ **Catatan Konsep:** Ketika subclass mendeklarasikan atribut dengan **nama yang sama** dengan atribut superclass, atribut superclass tersembunyi (*shadowing*). Pada kondisi inilah awalan **super.** menjadi penting untuk membedakan keduanya.

Pertanyaan Percobaan 3

1. Jelaskan fungsi **super** pada **super.phi = phi;** dan **super.r = r;** di method **setSuperPhi()** dan **setSuperR()** milik **Tabung**.
2. Jelaskan fungsi **super** dan **this** pada ekspresi **super.phi * super.r * super.r * this.t** di method **volume()**.
3. Mengapa **Tabung** tidak mendeklarasikan atribut **phi** dan **r**, tetapi tetap dapat mengaksesnya? Apa yang terjadi bila pada **Bangun** keduanya diubah menjadi **private**?
4. Pada Eksperimen 1, apakah output berubah ketika **super.phi** diganti **this.phi**? Jelaskan mengapa.
5. Pada Eksperimen 2, mengapa **r**, **this.r**, dan **super.r** menghasilkan nilai yang berbeda? Pada kondisi apa awalan **super.** menjadi **wajib** dipakai?

Percobaan 4: Konstruktor dan Multilevel Inheritance (ClassA, ClassB, ClassC)

Konstruktor **tidak diwariskan**, tetapi konstruktor superclass **selalu dijalankan lebih dulu** ketika objek subclass dibuat. Percobaan ini memakai tiga class berjenjang (*multilevel inheritance*): ClassB turunan ClassA, dan ClassC turunan ClassB.

Langkah 1: Diagram kelas

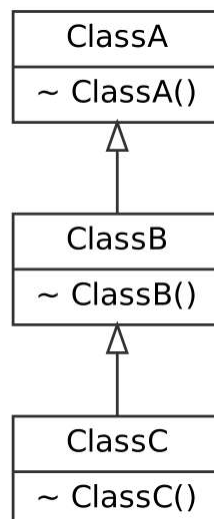


Figure 3: Multilevel inheritance: ClassB adalah subclass dari ClassA sekaligus superclass dari ClassC.

■ **Catatan Konsep:** Sebuah subclass dapat menjadi superclass bagi class lain, sehingga ClassB berperan **ganda**. Pada hierarki seperti ini, objek **ClassC** juga membawa bagian **ClassB** dan **ClassA** di dalamnya.

Langkah 2: Buat ClassA, ClassB, dan ClassC

Buat package `id.ac.polinema.inheritance.percobaan4`, lalu buat tiga class. Masing-masing konstruktor hanya mencetak sebuah pesan:

```
1 package id.ac.polinema.inheritance.percobaan4;
2
3 public class ClassA {
4     ClassA() {
5         System.out.println("konstruktor A dijalankan");
6     }
7 }
```

Kode 17: ClassA.java: konstruktor default

```
1 package id.ac.polinema.inheritance.percobaan4;
2
3 public class ClassB extends ClassA {
4     ClassB() {
5         System.out.println("konstruktor B dijalankan");
6     }
7 }
```

Kode 18: ClassB.java: turunan ClassA

```
1 package id.ac.polinema.inheritance.percobaan4;
2
3 public class ClassC extends ClassB {
4     ClassC() {
5         System.out.println("konstruktor C dijalankan");
6     }
7 }
```

Kode 19: ClassC.java: turunan ClassB

Langkah 3: Buat MainPercobaan4 lalu jalankan

```
1 package id.ac.polinema.inheritance.percobaan4;
2
3 public class MainPercobaan4 {
4     public static void main(String[] args) {
5         ClassC test = new ClassC();
6     }
7 }
```

Kode 20: MainPercobaan4.java: hanya membuat satu objek ClassC

 **Checkpoint:** output menampilkan tiga baris berurutan:
konstruktor A dijalankan, konstruktor B dijalankan, lalu konstruktor C dijalankan.

Langkah 4: Modifikasi 1, tambahkan super() di baris pertama

Tambahkan `super();` pada baris pertama konstruktor `ClassC`, lalu jalankan kembali `MainPercobaan4`:

```
1 package id.ac.polinema.inheritance.percobaan4;
2
3 public class ClassC extends ClassB {
4     ClassC() {
5         super();
6         System.out.println("konstruktor C dijalankan");
7     }
8 }
```

Kode 21: ClassC.java (Modifikasi 1): super() di baris pertama

✓ **Checkpoint:** output identik dengan Langkah 3.

Langkah 5: Modifikasi 2, pindahkan `super()` ke baris kedua

Pindahkan `super();` ke baris kedua (setelah `System.out.println`), lalu kompilasi. Setelah selesai, kembalikan `super();` ke baris pertama.

```
1 package id.ac.polinema.inheritance.percobaan4;
2
3 public class ClassC extends ClassB {
4     ClassC() {
5         System.out.println("konstruktor C dijalankan");
6         super();
7     }
8 }
```

Kode 22: ClassC.java (Modifikasi 2): `super()` setelah `println`

■ **Catatan Konsep:** Kompilasi **gagal** dengan pesan `call to super must be first statement in constructor` pada **ClassC.java baris 6**. Pemanggilan `super(...)` wajib menjadi baris pertama di dalam konstruktor subclass.

Pertanyaan Percobaan 4

1. Sebutkan class yang berperan sebagai superclass dan subclass pada percobaan ini beserta alasannya. Mengapa `ClassB` disebut berperan ganda?
2. Program hanya membuat satu objek (`new ClassC()`), tetapi tiga baris tercetak. Jelaskan mengapa konstruktor `ClassA` dan `ClassB` ikut dijalankan.
3. Pada Modifikasi 1, mengapa output tidak berbeda dari sebelumnya meskipun `super();` ditambahkan secara eksplisit?
4. Pada Modifikasi 2 terjadi error. Aturan apa yang dilanggar, dan mengapa Java menetapkan aturan tersebut?
5. Tuliskan urutan proses (bernomor) yang terjadi ketika `new ClassC()` dieksekusi, dimulai dari pemanggilan konstruktor `ClassC` hingga seluruh output tercetak.

Percobaan 5: Konstruktor Berparameter dan Overriding (Komputer, Desktop, Laptop)

Bagaimana jika superclass hanya punya konstruktor **berparameter**? Subclass wajib memanggilnya dengan `super(...)` dan memberi argumen yang sesuai. Percobaan ini juga memperkenalkan **overriding**: subclass menulis ulang method warisan, lalu tetap memakai versi superclass lewat `super.showInfo()`.

Langkah 1: Diagram kelas

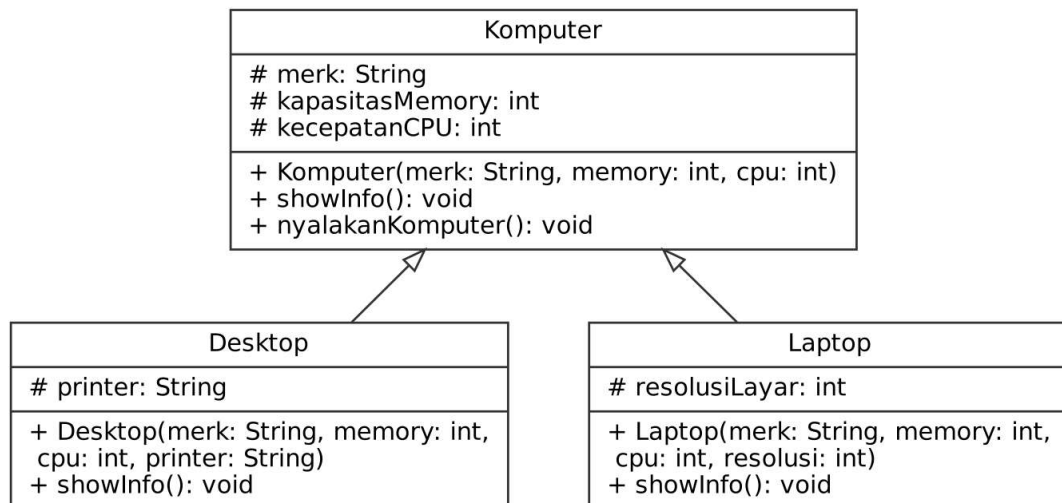


Figure 4: Komputer sebagai superclass, Desktop dan Laptop sebagai subclass (hierarchical inheritance: satu superclass, banyak subclass).

■ **Catatan Konsep:** Desktop adalah sebuah Komputer dan Laptop adalah sebuah Komputer (hubungan is-a). Keduanya berbagi atribut dan method yang sama dari Komputer, sehingga kode tidak ditulis berulang.

Langkah 2: Buat class Komputer

Buat package `id.ac.polinema.inheritance.percobaan5`, lalu buat class `Komputer`. Perhatikan bahwa konstruktornya **berparameter** dan tidak ada konstruktor default:

```
1 package id.ac.polinema.inheritance.percobaan5;
2
3 public class Komputer {
4     protected String merk;
5     protected int kapasitasMemory;
6     protected int kecepatanCPU;
7
8     public Komputer(String merk, int memory, int cpu) {
9         this.merk = merk;
10        this.kapasitasMemory = memory;
11        this.kecepatanCPU = cpu;
12    }
13
14    public void showInfo() {
15        System.out.println("Merk          : " + merk);
16        System.out.println("Kapasitas Memory: " + kapasitasMemory + " MB");
17        System.out.println("Kecepatan CPU   : " + kecepatanCPU + " MHz");
18    }
19
20    public void nyalakanKomputer() {
21        System.out.println("Komputer " + merk + " dinyalakan");
22    }
23 }
```

Kode 23: Komputer.java: konstruktor berparameter, method `showInfo()` dan `nyalakanKomputer()`

Langkah 3: Buat class Desktop dan Laptop

Buat dua subclass. Setiap konstruktor memanggil konstruktor `Komputer` dengan `super(...)`, dan setiap `showInfo()` menimpa (*override*) versi `Komputer`:

```
1 package id.ac.polinema.inheritance.percobaan5;
2
3 public class Desktop extends Komputer {
4     protected String printer;
5
6     public Desktop(String merk, int memory, int cpu, String printer) {
7         super(merk, memory, cpu);
8         this.printer = printer;
9     }
10
11     @Override
12     public void showInfo() {
13         super.showInfo();
14         System.out.println("Printer      : " + printer);
15     }
16 }
```

Kode 24: Desktop.java: super(...), @Override, dan super.showInfo()

```
1 package id.ac.polinema.inheritance.percobaan5;
2
3 public class Laptop extends Komputer {
4     protected int resoliLayar;
5
6     public Laptop(String merk, int memory, int cpu, int resoli) {
7         super(merk, memory, cpu);
8         this.resoliLayar = resoli;
9     }
10
11     @Override
12     public void showInfo() {
13         super.showInfo();
14         System.out.println("Resoli Layar : " + resoliLayar + "p");
15     }
16 }
```

Kode 25: Laptop.java

Langkah 4: Buat MainPercobaan5 lalu jalankan

```
1 package id.ac.polinema.inheritance.percobaan5;
2
3 public class MainPercobaan5 {
4     public static void main(String[] args) {
5         Desktop desk = new Desktop("Dell", 2048, 3500, "Canon");
6         Laptop lap = new Laptop("Asus", 4096, 2500, 720);
7
8         desk.showInfo();
9         System.out.println();
10        lap.showInfo();
11        System.out.println();
12        desk.nyalakanKomputer();
13    }
14 }
```

Kode 26: MainPercobaan5.java

✓ **Checkpoint:** output menampilkan data Desktop (Merk : Dell hingga Printer : Canon), data Laptop (Merk : Asus hingga Resolusi Layar : 720p), lalu Komputer Dell dinyalakan.

Langkah 5: Eksperimen 1, hapus pemanggilan super(...)

Hapus baris `super(merk, memory, cpu);` pada konstruktor `Desktop`, lalu kompilasi. Setelah selesai, kembalikan barisnya.

■ **Catatan Konsep:** Kompilasi gagal pada `Desktop.java` baris 6 dengan pesan `constructor Komputer in class Komputer cannot be applied to given types (required: String,int,int, found: no arguments)`. Tanpa `super(...)`, compiler menyisipkan `super();` otomatis, tetapi `Komputer` tidak memiliki konstruktor tanpa parameter.

Langkah 6: Eksperimen 2, salah nama method override

Pada `Desktop`, ubah nama method menjadi `showinfo()` (huruf "i" kecil) dengan anotasi `@Override` tetap ada, lalu kompilasi. Kemudian hapus anotasi `@Override`, kompilasi lagi, dan jalankan `MainPercobaan5`. Setelah selesai, kembalikan seperti semula.

```
1 package id.ac.polinema.inheritance.percobaan5;
2
3 public class Desktop extends Komputer {
4     protected String printer;
5
6     public Desktop(String merk, int memory, int cpu, String printer) {
7         super(merk, memory, cpu);
8         this.printer = printer;
9     }
10
11     public void showinfo() {
12         super.showInfo();
13         System.out.println("Printer : " + printer);
14     }
15 }
```

Kode 27: `Desktop.java` (Eksperimen 2): nama method salah ketik, anotasi `@Override` dihapus

■ **Catatan Konsep:** Dengan `@Override`, compiler menolak kode dengan pesan `method does not override or implement a method from a supertype`. Tanpa `@Override`, kompilasi lolos, tetapi `showinfo()` hanya menjadi method baru, sehingga `desk.showInfo()` memanggil versi `Komputer` dan baris `Printer` tidak tercetak.

Pertanyaan Percobaan 5

1. Jelaskan fungsi `super(merk, memory, cpu)` pada konstruktor `Desktop`. Atribut apa saja yang diisi oleh baris tersebut, dan atribut apa yang diisi oleh baris berikutnya?
2. Pada Eksperimen 1, mengapa error muncul di sini, padahal pada Percobaan 4 `super()` juga tidak ditulis tetapi program tetap berjalan?
3. Method `showInfo()` ditulis di `Komputer` sekaligus di `Desktop`. Apa istilah untuk kondisi ini? Apa yang tercetak bila baris `super.showInfo();` pada `Desktop` dihapus?

4. Pada Eksperimen 2, jelaskan perbedaan hasil kompilasi dengan dan tanpa `@Override`. Apa manfaat menuliskan `@Override`?
5. **Tantangan.** Buat class `Workstation` sebagai turunan `Desktop` dengan atribut `gpu` (String). Class ini harus menimpa `showInfo()` sehingga menampilkan seluruh informasi `Desktop` ditambah baris GPU. Ketika `new Workstation(...)` dibuat, konstruktor class apa saja yang terpanggil, dan dalam urutan apa?

D. Tugas dan Deliverable

Kumpulkan hal berikut sesuai format yang diminta Dosen:

- Screenshot output program untuk **setiap** Percobaan (1 sampai 5), termasuk pesan error yang diminta pada pertanyaan.
- Source code seluruh package `id.ac.polinema.inheritance.percobaan1` s.d. `percobaan5`, serta package tugas mandiri.

Tugas mandiri:

Tugas 1: Pegawai, Dosen, dan DaftarGaji

Buat program pada package `id.ac.polinema.inheritance.tugas1` berdasarkan diagram kelas berikut. `Dosen` mewarisi `Pegawai`, sedangkan `DaftarGaji` menyimpan kumpulan `Pegawai` dalam sebuah array (agregasi, seperti pada Jobsheet 04).

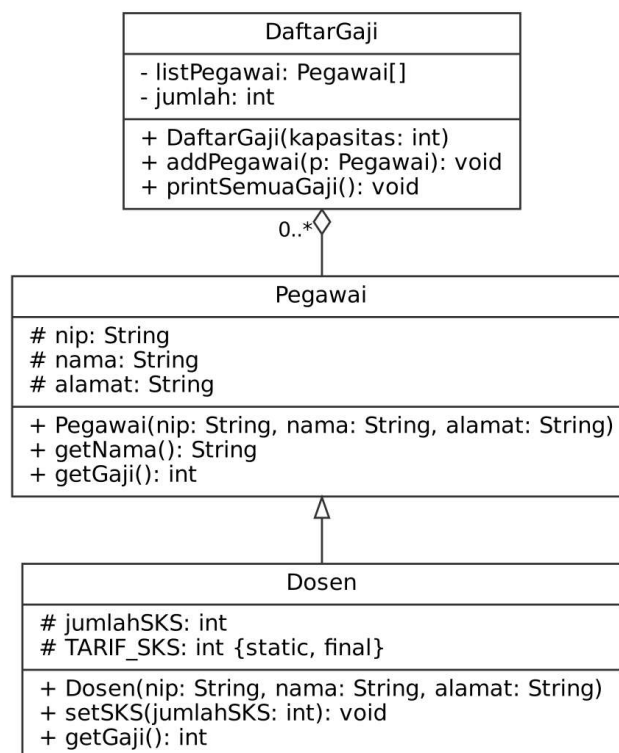


Figure 5: Diagram kelas Tugas 1, `Dosen` mewarisi `Pegawai`, `DaftarGaji` berelasi agregasi dengan `Pegawai`.

- Gaji `Pegawai` adalah gaji pokok tetap **Rp1.500.000** (`getGaji()` mengembalikan `1500000`).
- Gaji `Dosen` adalah gaji pokok `Pegawai` ditambah `jumlahSKS × TARIF_SKS`, dengan `TARIF_SKS` berupa konstanta bernilai **100000**. `getGaji()` pada `Dosen` harus menimpa versi `Pegawai` dengan `@Override` dan memanfaatkan `super.getGaji()`.

- `DaftarGaji(int)` menerima kapasitas array. `addPegawai(Pegawai)` menambahkan objek ke array, dan `printSemuaGaji()` mencetak nama dan gaji setiap pegawai yang sudah ditambahkan.
- Access modifier mengikuti diagram: atribut `Pegawai` dan `Dosen` bersifat `protected`, atribut `DaftarGaji` bersifat `private`.
- Buat class `MainTugas1` yang menambahkan satu `Pegawai` (NIP "P001", Budi, Malang) dan satu `Dosen` (NIP "D001", Siti, Surabaya, 12 SKS), lalu memanggil `printSemuaGaji()`.

☑ **Checkpoint:** dari data di atas, output menampilkan
 Budi : 1500000
 dan Siti : 2700000.

Pertanyaan analisis: (a) Array `Pegawai[]` dapat menampung objek `Dosen`. Mengapa hal itu diperbolehkan? (b) Ketika `printSemuaGaji()` memanggil `getGaji()` pada objek `Dosen`, versi method milik class mana yang dijalankan?

Tugas 2: Televisi dan TelevisiModern

Buat program pada package `id.ac.polinema.inheritance.tugas2` berdasarkan diagram kelas berikut.

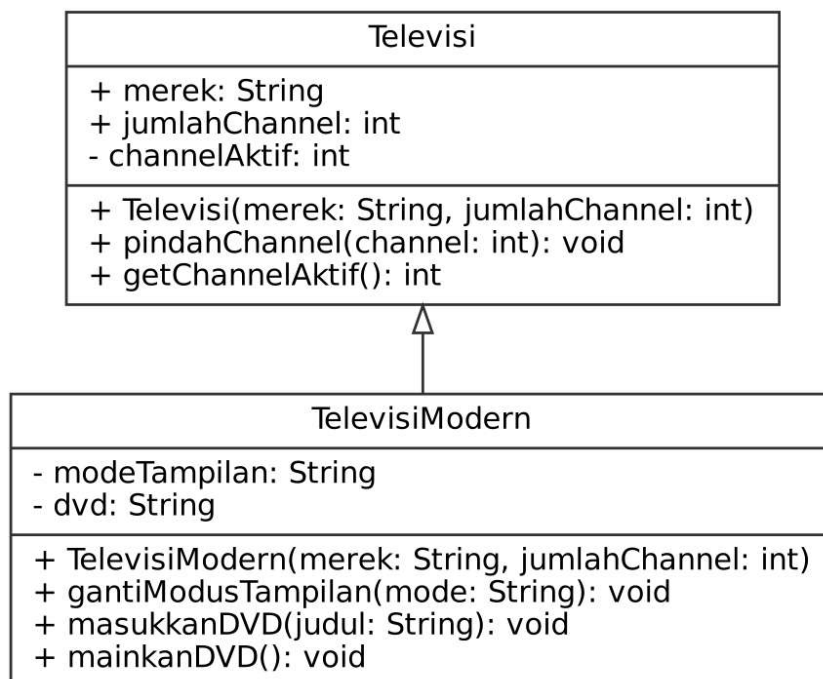


Figure 6: Diagram kelas Tugas 2, `TelevisiModern` mewarisi `Televisi`.

- Channel aktif awal adalah 1. `pindahChannel(int)` hanya mengubah channel bila nilainya berada pada rentang 1 sampai `jumlahChannel`.
- `channelAktif` bersifat `private`, sehingga `TelevisiModern` maupun `MainTugas2` hanya dapat membacanya lewat `getChannelAktif()`.
- `mainkanDVD()` mencetak `Sedang memainkan DVD:` diikuti judul DVD. Sebelum ada DVD yang dimasukkan, judulnya adalah `kosong`.
- Konstruktor `TelevisiModern` harus memanggil konstruktor `Televisi` dengan `super(...)`.

Uji dengan class **MainTugas2** berikut:

```
1 package id.ac.polinema.inheritance.tugas2;
2
3 public class MainTugas2 {
4     public static void main(String[] args) {
5         TelevisiModern tv = new TelevisiModern("Samsung", 100);
6         System.out.println("Channel aktif: " + tv.getChannelAktif());
7         tv.pindahChannel(20);
8         System.out.println("Channel aktif sekarang: "
9             + tv.getChannelAktif());
10        tv.gantiModusTampilan("HDMI");
11        tv.mainkanDVD();
12        tv.masukkanDVD("The Matrix");
13        tv.mainkanDVD();
14    }
15 }
```

Kode 28: MainTugas2.java: program uji untuk Tugas 2

☑ **Checkpoint:** output menampilkan

Channel aktif: 1, Channel aktif sekarang: 20,

Sedang memainkan DVD: kosong, dan Sedang memainkan DVD: The Matrix.

Uji tambahan: panggil `tv.pindahChannel(150)` lalu cetak channel aktif. Tuliskan hasilnya dan jelaskan mengapa `channelAktif` tidak dapat diubah langsung dari **MainTugas2**.

Tugas 3 (opsional, pengayaan): Karakter Game

Pada package `id.ac.polinema.inheritance.pengayaan`, bangun hierarki karakter game berikut. **Character** adalah superclass, sedangkan **Angel**, **Human**, dan **Wizard** adalah subclass.

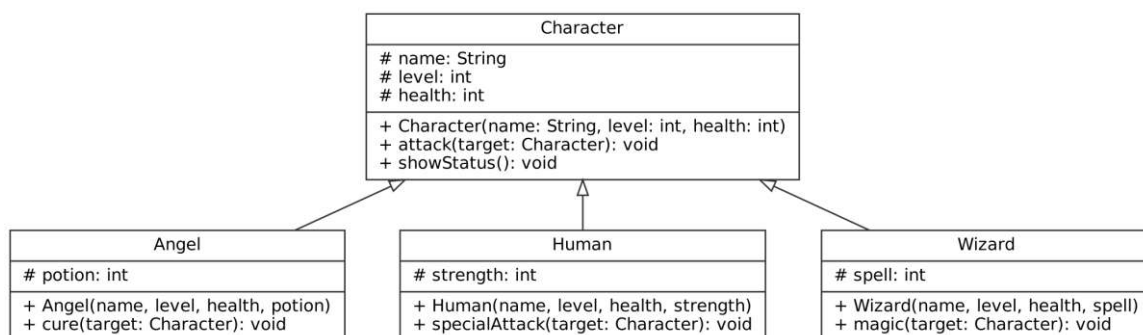


Figure 7: Diagram kelas karakter game, satu superclass dengan tiga subclass.

- `attack(target)` mengurangi `health` target sebesar 10, dan `showStatus()` mencetak nama, level, dan health.
- `Angel.cure(target)` mengembalikan health target menjadi 100 dan mengurangi `potion` milik `Angel` sebanyak 1.
- `Human.specialAttack(target)` mengurangi health target sebesar `10 + strength`. Strength tidak berkurang.
- `Wizard.magic(target)` mengurangi health target sebesar 50 dan mengurangi `spell` milik `Wizard` sebanyak 1.

 **Info:** Nama **Character** menempa class bawaan **java.lang.Character** di dalam project Anda. Nama ini dipertahankan agar sesuai dengan slide, tetapi pada proyek nyata gunakan nama lain (misalnya **Karakter**).

Uji dengan class **MainTugas3** berikut:

```
1 package id.ac.polinema.inheritance.pengayaan;
2
3 public class MainTugas3 {
4     public static void main(String[] args) {
5         Angel esther = new Angel("Esther", 10, 100, 5);
6         Human jackal = new Human("Jackal", 13, 100, 7);
7         Wizard quistis = new Wizard("Quistis", 20, 100, 3);
8
9         System.out.println("Begin game...");
10        esther.showStatus();
11        jackal.showStatus();
12        quistis.showStatus();
13
14        System.out.println("Jackal special attack to quistis, "
15            + "quistis cast magic to jackal,");
16        System.out.println("esther cure jackal, quistis attack esther...");
17        jackal.specialAttack(quistis);
18        quistis.magic(jackal);
19        esther.cure(jackal);
20        quistis.attack(esther);
21
22        esther.showStatus();
23        jackal.showStatus();
24        quistis.showStatus();
25    }
26 }
```

Kode 29: MainTugas3.java: program uji untuk Tugas 3

 **Checkpoint:** setelah serangan, **Esther** berhealth **90**, **Jackal** kembali **100** (karena disembuhkan **Esther**), dan **Quistis** berhealth **83** (= 100 – (10 + 7)).

Tugas 4: Jawab singkat

1. Jelaskan dengan bahasa Anda sendiri perbedaan hubungan **is-a** (inheritance) dan **has-a** (aggregation/composition), lalu beri satu contoh masing-masing dari jobsheet ini.
2. Ringkas aturan pewarisan untuk tiga hal berikut dalam 3–5 kalimat: member **private**, member **protected**, dan konstruktor.

 **Info:** Tugas 1 sengaja menggabungkan **inheritance** (**Dosen** adalah sebuah **Pegawai**) dan **aggregation** (**DaftarGaji** memiliki kumpulan **Pegawai**), supaya Anda dapat membedakan keduanya dalam satu program.

E. Kriteria Penilaian

Komponen	Bobot	Kriteria Lengkap (100%)	Kriteria Minimum
Langkah kerja Percobaan 1-5 tuntas	35%	Seluruh langkah dijalankan dan berfungsi sesuai checkpoint	Sebagian besar langkah selesai, hasil akhir tiap Percobaan berjalan
Checkpoint terverifikasi	20%	Semua checkpoint tercapai dan dibuktikan (screenshot/output)	Sebagian checkpoint terbukti
Pertanyaan pemahaman terjawab	20%	Seluruh pertanyaan terjawab dengan alasan tepat dan merujuk baris kode yang relevan	Sebagian besar pertanyaan terjawab, sebagian alasan kurang tepat
Tugas mandiri	25%	Tugas 1 dan 2 benar sesuai diagram dan output, Tugas 4 terjawab dengan tepat	Tugas 1 dan 2 ada meski output belum sesuai atau jawaban belum lengkap